



MongoDB Cheat Sheet (Basic to Advanced)

Last Updated : 15 Jul, 2024

In this **MongoDB** cheat sheet, we'll learn about the MongoDB key concepts, including data types, CRUD operations, query techniques, aggregation framework, indexing strategies, transaction support, and data modeling approaches. Each section is explained with examples and explanations.

MongoDB is a **document-oriented** NoSQL database that revolutionizes data storage with its **flexibility** and **scalability**. By storing data in **JSON-like** documents, MongoDB offers developers a powerful and intuitive way to handle complex data structures. From basic **CRUD** operations to advanced **aggregation** techniques, MongoDB empowers users to build robust and **dynamic** applications.



Basics of MongoDB

What is MongoDB	MongoDB is a document-oriented NoSQL database that stores data in flexible, JSON-like documents.
DataTypes in MongoDB	MongoDB supports various data types including string, integer, double, boolean, arrays, objects, dates, and null.
What is ObjectId in MongoDB	ObjectId is a 12-byte hexadecimal number that uniquely identifies documents in a collection.
MongoDB Atlas	MongoDB Atlas is a fully managed cloud database service . It provides automated backups, monitoring, and security features.
MongoDB Compass	MongoDB Compass is a graphical user interface for MongoDB . It allows users to visualize data, run queries, and analyze performance.
What is MongoDB Shell	MongoDB Shell is a command-line interface for interacting with MongoDB instances . It allows users to execute queries, perform administrative tasks, and manage databases .

CRUD Operations in MongoDB

This section covers the basics of working with your MongoDB database using [CRUD operations](#). You'll learn how to **Create**, **Read**, **Update**, and **Delete** documents which allowing you to efficiently manage your data. Get ready to add, retrieve, modify, and remove information easily



Fitten Code: Code Faster, Code Better

Connect to MongoDB

```
mongo
```

Open a terminal and start the MongoDB shell by typing mongo.

Create and Use a Database

```
use blog
```

Create (if not exists) and use the 'blog' database

Create Collections

```
// Create a 'posts' collection  
db.createCollection("posts")
```

```
// Create a 'users' collection  
db.createCollection("users")
```

Create two collections: posts for storing blog posts and users for storing user information.

Insert Operations

```
db.posts.insertOne({
  title: "Introduction to MongoDB",
  content: "MongoDB is a NoSQL database.",
  author: "John Doe",
  tags: ["mongodb", "nosql", "database"]
})
```

Insert multiple documents into 'users' collection

```
db.users.insertMany([
  {
    username: "johndoe",
    email: "johndoe@example.com",
    age: 30
  },
  {
    username: "janedoe",
    email: "janedoe@example.com",
    age: 28
  }
])
```

Update Operations

Update a document in 'users' collection

```
db.users.updateOne(
  { username: "johndoe" },
  { $set: { age: 31 } }
)
```

Update multiple documents in 'posts' collection

```
db.posts.updateMany(
  { tags: "mongodb" },
  { $addToSet: { tags: "database" } }
```

Delete Operations

Delete a document from 'users' collection

```
db.users.deleteOne({ username: "janedoe" })
```

Delete multiple documents from 'posts' collection

```
db.posts.deleteMany({ author: "John Doe" })
```

Drop the entire 'users' collection

```
db.users.drop()
```

Query Operations

Find all documents in 'posts' collection

```
db.posts.find()
```

Find one document in 'posts' collection

```
db.posts.findOne({ title: "Introduction to MongoDB" })
```

Find and modify a document in 'posts' collection

```
db.posts.findOneAndUpdate(  
  { title: "Introduction to MongoDB" },  
  { $set: { content: "MongoDB is a flexible and scalable NoSQL  
database." } }  
)
```

Find one and delete a document in 'posts' collection

```
db.posts.findOneAndDelete({ author: "John Doe" })
```

Find one and replace a document in 'posts' collection

```
{ title: "MongoDB Overview", content: "A detailed guide to MongoDB."
}
```

Query with Projections

Find documents with projection (only return 'title' and 'author' fields)

```
db.posts.find({}, { title: 1, author: 1 })
```

Query nested documents (e.g., find users with email ending in '.com')

```
db.users.find({ "email": /\.*\..com$/ })
```

Query documents with null or missing fields

```
db.users.find({ email: null })
```

Show Database Information

```
// Show available databases
show dbs
```

```
// Show collections in the current database
show collections
```

To see a list of available databases and their collections

MongoDB Operators

MongoDB operators are like tools that help you work with your data effectively. They allow you to find specific information, make changes and analyze your data in MongoDB.

Mastering these operators gives you the ability to manage and explore your data more efficiently, uncovering valuable insights along the way.

1. Comparison Operators

```
db.users.find({ age: { $gt: 30 } })
```

Find documents where age is less than or equal to 28 in 'users' collection

```
db.users.find({ age: { $lte: 28 } })
```

Find documents where title is equal to "MongoDB Overview" in 'posts' collection

```
db.posts.find({ title: { $eq: "MongoDB Overview" } })
```

Find documents where age is not equal to 30 in 'users' collection

```
db.users.find({ age: { $ne: 30 } })
```

In these queries, we utilize the \$gt (greater than), \$lt (less than), and \$eq (equality) comparison operators to filter documents based on specific criteria. Additionally, we demonstrate the \$ne (not equal) operator to find documents where a field does not match a specified value.

2. Logical Operators

Find documents where age is greater than 25 AND less than 35 in 'users' collection

```
db.users.find({ $and: [ { age: { $gt: 25 } }, { age: { $lt: 35 } } ] })
```

Find documents where username is "johndoe" OR email is "janedoe@example.com" in 'users' collection

```
db.users.find({ $or: [ { username: "johndoe" }, { email: "janedoe@example.com" } ] })
```

Find documents where age is NOT equal to 30 in 'users' collection

```
db.users.find({ age: { $not: { $eq: 30 } } })
```

```
db.users.find({ age: { $nor: [ { $eq: 30 }, { $eq: 31 } ] } })
```

We use the `$and` operator to find documents where multiple conditions must be satisfied simultaneously. The `$or` operator is utilized to find documents where at least one of the specified conditions is met. Using the `$not` operator, we exclude documents where a specific condition is true. The `$nor` operator is used to find documents where none of the specified conditions are met.

3. Arithmetic Operators

Add 5 to the age of all users in 'users' collection

```
db.users.updateMany({}, { $add: { age: 5 } })
```

Subtract 2 from the age of users aged 30 in 'users' collection

```
db.users.updateMany({ age: 30 }, { $subtract: { age: 2 } })
```

Multiply the age of users by 2 in 'users' collection

```
db.users.updateMany({}, { $multiply: { age: 2 } })
```

Divide the age of all users by 2 in 'users' collection

```
db.users.updateMany({}, { $divide: { age: 2 } })
```

Calculate the absolute value of the age of all users in 'users' collection

```
db.users.updateMany({}, { $abs: { age: true } })
```

We use the `$add`, `$subtract`, `$multiply`, and `$divide` operators to perform addition, subtraction, multiplication, and division respectively on numeric fields. The `$abs` operator calculates the absolute value of numeric fields.

4. Field Update Operators

Update the age of users to the maximum value of 40 in 'users' collection

Update the age of users to the minimum value of 20 in 'users' collection

```
db.users.updateMany({}, { $min: { age: 20 } })
```

Increment the age of users by 1 in 'users' collection

```
db.users.updateMany({}, { $inc: { age: 1 } })
```

Multiply the age of users by 1.1 in 'users' collection

```
db.users.updateMany({}, { $mul: { age: 1.1 } })
```

We use the \$max and \$min operators to update fields to the maximum or minimum value respectively. The \$inc operator increments numeric fields by a specified value. The \$mul operator multiplies numeric fields by a specified value.

5. Array Expression Operators

Find documents where 'tags' field is an array in 'posts' collection

```
db.posts.find({ tags: { $isArray: true } })
```

Find documents in 'posts' collection where the size of the 'tags' array is 3

```
db.posts.find({ $expr: { $eq: [{ $size: "$tags" }, 3] } })
```

Find the first element of the 'tags' array in each document of 'posts' collection

```
db.posts.aggregate([  
  ... { $project: { firstTag: { $arrayElemAt: ["$tags", 0] } } }  
])
```

Concatenate the 'tags' arrays of all documents in 'posts' collection

```
db.posts.aggregate([  
  ... { $group: { _id: null, allTags: { $concatArrays: "$tags" } } }  
])
```

```
db.posts.updateMany({}, { $reverseArray: "$tags" })
```

We use the [\\$isArray](#) operator to find documents where a field is an array. The [\\$size](#) operator is used to find documents based on the size of an array field. With [\\$arrayElemAt](#), we retrieve a specific element from an array field. The [\\$concatArrays](#) operator concatenates arrays. Finally, [\\$reverseArray](#) reverses the elements of an array.

6. Array Update Operators

Remove all occurrences of “**mongodb**” from the ‘tags’ array in ‘posts’ collection

```
db.posts.updateMany({}, { $pull: { tags: "mongodb" } })
```

Remove the last element from the ‘tags’ array in all documents of ‘posts’ collection

```
db.posts.updateMany({}, { $pop: { tags: 1 } })
```

Remove all occurrences of “**nosql**” and “**database**” from the ‘tags’ array in ‘posts’ collection

```
db.posts.updateMany({}, { $pullAll: { tags: ["nosql", "database"] } })
```

Add “**newtag**” to the end of the ‘tags’ array in a specific document in ‘posts’ collection

```
db.posts.updateOne({ title: "Introduction to MongoDB" }, { $push: { tags: "newtag" } })
```

Update the ‘tags’ array in all documents where “**mongodb**” is present with “**updatedtag**”

```
db.posts.updateMany({ tags: "mongodb" }, { $set: { "tags.$": "updatedtag" } })
```

Concatenate the 'title' and 'content' fields into a new field '**fullText**' in 'posts' collection

```
db.posts.aggregate([
  {
    $project: {
      fullText: { $concat: ["$title", " ", "$content"] }
    }
  }
])
```

Compare the 'title' field case insensitively to "**MongoDB**" in 'posts' collection

```
db.posts.find({ $expr: { $eq: [{ $strcasecmp: ["$title", "MongoDB"] },
  {} ] } })
```

Convert the '**title**' field to **uppercase** in 'posts' collection

```
db.posts.updateMany({}, { $set: { title: { $toUpper: "$title" } } })
```

Convert the '**title**' field to **lowercase** in 'posts' collection

```
db.posts.updateMany({}, { $set: { title: { $toLower: "$title" } } })
```

Extract the first 5 characters from the '**title**' field in 'posts' collection

```
db.posts.aggregate([
  { $project: { firstFiveChars: { $substrCP: ["$title", 0, 5] } } }
])
```

We use the [\\$concat](#) operator to concatenate fields or strings. [\\$strcasecmp](#) compares strings case insensitive. The [\\$toUpper](#) operator converts a string to uppercase. [\\$toLower](#) converts a string to lowercase. [\\$substrCP](#) extracts a substring from a string based on code points.

MongoDB Aggregation Framework

We'll perform various [aggregation](#) operations using MongoDB's aggregation framework

Update documents with aggregation pipeline: multiply 'age' field by 2 and store in 'doubleAge' field

```
db.users.aggregate([
  { $addFields: { doubleAge: { $multiply: ["$age", 2] } } },
  { $out: "users" }
])
```

Count the number of documents in 'users' collection

```
db.users.aggregate([
  { $count: "total_users" }
])
```

Group documents in 'users' collection by 'age' and calculate the count in each group

```
db.users.aggregate([
  { $group: { _id: "$age", count: { $sum: 1 } } }
])
```

Perform a left outer join between 'posts' and 'users' collections based on 'author' field

```
db.posts.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "author",
      foreignField: "username",
      as: "author_info"
    }
  }
])
```

Get the first document in each group sorted by 'age' in descending order in 'users' collection

```
... { $group: { _id: null, oldestUser: { $first: "$$ROOT" } } }  
])
```

Perform map-reduce operation to calculate the total age of all users

```
var mapFunction = function () {  
    emit("totalAge", this.age);  
};  
  
var reduceFunction = function (key, values) {  
    return Array.sum(values);  
};  
  
db.users.mapReduce(  
    mapFunction,  
    reduceFunction,  
    { out: { inline: 1 } }  
);
```

We use various stages such as **\$addFields**, **\$out**, **\$count**, **\$group**, **\$lookup**, **\$first**, and map-reduce for different aggregation operations.

Aggregation framework allows us to perform complex computations, transformations, and data analysis on MongoDB collections efficiently.

MongoDB Indexing

Indexing enhances query performance and allows for efficient data retrieval in MongoDB

Create a **single field index** on the 'username' field in the 'users' collection

```
db.users.createIndex({ username: 1 })
```

Get the list of indexes on the 'users' collection

```
db.users.getIndexes()
```

Drop the index on the 'username' field in the 'users' collection

Create a [compound index](#) on the 'title' and 'content' fields in the 'posts' collection

```
db.posts.createIndex({ title: 1, content: 1 })
```

Create a [multikey index](#) on the 'tags' array field in the 'posts' collection

```
db.posts.createIndex({ tags: 1 })
```

Create a [text index](#) on the 'content' field in the 'posts' collection

```
db.posts.createIndex({ content: "text" })
```

Create a [unique index](#) on the 'email' field in the 'users' collection

```
db.users.createIndex({ email: 1 }, { unique: true })
```

We use `createIndex()` to create various types of indexes, such as single field, compound, multikey, text, and unique indexes. `getIndexes()` retrieves the list of indexes on a collection. `dropIndex()` drops an index by its name.

Transactions in MongoDB

MongoDB supports multi-document [ACID transactions](#), allowing for **atomicity**, **consistency**, **isolation**, and **durability**.

```
// Start a session
session = db.getMongo().startSession()

// Start a transaction
session.startTransaction()

try {
    // Perform operations within the transaction
    db.collection1.insertOne({ field1: "value1" }, { session: session })
    db.collection2.updateOne({ field2: "value2" }, { $set: { field3:
"value3" } }, { session: session })

    // Commit the transaction
```

```
..... // Abort the transaction on error
..... session.abortTransaction()
}
```

Data Modeling in MongoDB

Data modeling in MongoDB involves designing schemas and relationships between documents.

```
// Relationship: Embedding data in documents
db.users.insertOne({
  ..... username: "john_doe",
  ..... email: "john@example.com",
  ..... posts: [
  .....     { title: "Post 1", content: "Content 1" },
  .....     { title: "Post 2", content: "Content 2" }
  ..... ]
  .....)
})
```

```
// Relationship: Referencing documents
db.comments.insertOne({
  ..... user_id: ObjectId("user_id_here"),
  ..... post_id: ObjectId("post_id_here"),
  ..... content: "Comment content"
  .....)
})
```

```
// Specify JSON schema validation
db.createCollection("collection_name", {
  ..... validator: {
  .....     $jsonSchema: {
  .....         bsonType: "object",
  .....         required: ["field1", "field2"],
  .....         properties: {
  .....             field1: {
  .....                 bsonType: "string"
  .....             },
  .....             field2: {
  .....                 bsonType: "number"
  .....             }
  .....         }
  .....     }
  ..... }
```

```
    }  
  })
```

```
// Scaling in MongoDB involves sharding, replication, and proper index  
usage to distribute data across multiple servers.
```

We demonstrate embedding data in documents and referencing documents to model relationships between collections. [JSON](#) schema validation ensures data integrity by enforcing structure and data types. Scaling in **MongoDB** involves strategies like sharding and replication to handle large volumes of data.

Conclusion

Overall, MongoDB stands out as a leading NoSQL database with its flexible document-oriented approach and powerful features like CRUD operations, indexing and the aggregation framework. Whether you're handling large-scale data or building dynamic applications, MongoDB is easy to use and scalability make it a preferred choice for modern development. By mastering MongoDB's capabilities, developers can leverage its strengths to create robust, efficient and scalable solutions that meet the demands of today's data-driven world.

Want to be a Software Developer or a Working Professional looking to enhance your **Software Development Skills**? Then, master the concepts of Full-Stack Development. Our [Full Stack Development - React and Node.js Course](#) will help you achieve this quickly. Learn everything from **Front-End to Back-End Development** with hands-on Projects and real-world examples. This course enables you to build scalable, efficient, dynamic web applications that stand out. Ready to become an expert in Full-Stack? Enroll Now and Start Creating the Future!

v vivek... + Follow



1

Next Article

Similar Reads

SQL Cheat Sheet (Basic to Advanced)

Creating and managing databases in SQL involves various commands and concepts that handle the structuring, querying, and manipulation of data. In this...

15 min read

30 Days of SQL - From Basic to Advanced Level

This basic to advanced SQL tutorial covers the entire SQL syllabus in a structured way and provides the best learning material and strategies to master complete...

8 min read

Connect MongoDB (AWS) from Local Machine using WinSCP and MongoDB...

Pre-requisite: AWS and MongoDB In this article, we are going to connect to the Mongo database of the AWS Ubuntu Server instance using WinSCP and learn...

4 min read

How to Integrate MongoDB Atlas and Segment using MongoDB Stitch

Data integration has become a crucial component of modern business strategies, allowing companies to enhance the power of data for informed decision-making....

5 min read

MongoDB - Delete Multiple Documents Using MongoDB Shell

The `db.collection.deleteMany()` method is used to delete multiple documents from a collection in Mongo Shell. This method deletes multiple documents from...

2 min read

Connect MongoDB Atlas Cluster to MongoDB Compass

MongoDB Compass is a free GUI for MongoDB. You might want to connect MongoDB Atlas Cluster to MongoDB Compass to take benefit of the GUI model...

4 min read

MongoDB Shell vs MongoDB Node JS Driver

4 min read

MongoDB Compass vs MongoDB Atlas

MongoDB is a popular NoSQL database known for flexibility and scalability. MongoDB Compass offers a graphical interface for managing databases visually...

5 min read

Securing Elasticsearch with Advanced SSL/TLS Encryption Configuration

Securing Elasticsearch is crucial for protecting your data and ensuring secure communication within your Elasticsearch cluster and between clients. One of th...

5 min read

MongoDB vs MySQL

Differences between MongoDB and MySQL on various parameters are discussed below: What are MongoDB and MySQL? MongoDB MySQL MongoDB is an ope...

2 min read

Difference between Oracle and MongoDB

1. Oracle : Oracle is a relational database management system (RDBMS). It was developed by Oracle Corporation in 1980. It is the first database designed for gr...

2 min read

Connecting MongoDB to Jupyter Notebook

MongoDB is one of the most famous NoSql databases. It is an open-source database. The simple meaning of the NoSql database is a non-relational...

3 min read

Introduction about Node.js and MongoDB

Databases SQL MySQL PostgreSQL PL/SQL MongoDB SQL Cheat Sheet SQL Interview Questions My!

a non-tabular format. Depending on the format of the data stored, NoSQL...

4 min read

MongoDB | Delete Database using MongoShell

2 min read

Create a Newsletter Sourcing Data using MongoDB

There are many news delivery websites available like ndtv.com. In this article, let us see the very useful and interesting feature of how to get the data from...

7 min read

Difference between RDBMS and MongoDB

MongoDB is an open-source document-oriented database used for high volume data storage. It falls under the classification of a NoSQL database. NoSQL tool...

2 min read

How MongoDB works ?

MongoDB is an open-source document-oriented database. It is used to store a larger amount of data and also allows you to work with that data. MongoDB is...

3 min read

MongoDB - Embedded Documents

MongoDB provides you a cool feature which is known as Embedded or Nested Document. Embedded document or nested documents are those types of...

2 min read

MongoDB - Insert Single Document Using MongoShell

In MongoDB, insert operations are used to add new documents in the collection. If the collection does not exist, then the insert operations create the collection b...

2 min read

MongoDB - Insert Multiple Document Using MongoShell

In MongoDB, insert operations are used to add new documents in the collection. If the collection does not exist, then the insert operations create the collection b...

2 min read



Corporate & Communications Address:-
A-143, 9th Floor, Sovereign Corporate
Tower, Sector- 136, Noida, Uttar Pradesh
(201305) | Registered Address:- K 061,
Tower K, Gulshan Vivante Apartment,
Sector 137, Noida, Gautam Buddh
Nagar, Uttar Pradesh, 201305



Company

About Us
Legal
Careers
In Media
Contact Us
Advertise with us
GFG Corporate Solution
Placement Training Program

Languages

Python
Java

Explore

Job-A-Thon Hiring Challenge
Hack-A-Thon
GfG Weekly Contest
Offline Classes (Delhi/NCR)
DSA in JAVA/C++
Master System Design
Master CP
GeeksforGeeks Videos
Geeks Community

DSA

Data Structures
Algorithms

[SQL](#)[R Language](#)[Android Tutorial](#)[DSA Interview Questions](#)[Competitive Programming](#)

Data Science & ML

[Data Science With Python](#)[Data Science For Beginner](#)[Machine Learning](#)[ML Maths](#)[Data Visualisation](#)[Pandas](#)[NumPy](#)[NLP](#)[Deep Learning](#)

Python Tutorial

[Python Programming Examples](#)[Django Tutorial](#)[Python Projects](#)[Python Tkinter](#)[Web Scraping](#)[OpenCV Tutorial](#)[Python Interview Question](#)

DevOps

[Git](#)[AWS](#)[Docker](#)[Kubernetes](#)[Azure](#)[GCP](#)[DevOps Roadmap](#)

School Subjects

[Mathematics](#)[Physics](#)[Chemistry](#)[Biology](#)[Social Science](#)[English Grammar](#)

Databases

[SQL](#)[MySQL](#)

Web Technologies

[HTML](#)[CSS](#)[JavaScript](#)[TypeScript](#)[ReactJS](#)[NextJS](#)[NodeJs](#)[Bootstrap](#)[Tailwind CSS](#)

Computer Science

[GATE CS Notes](#)[Operating Systems](#)[Computer Network](#)[Database Management System](#)[Software Engineering](#)[Digital Logic Design](#)[Engineering Maths](#)

System Design

[High Level Design](#)[Low Level Design](#)[UML Diagrams](#)[Interview Guide](#)[Design Patterns](#)[OOAD](#)[System Design Bootcamp](#)[Interview Questions](#)

Commerce

[Accountancy](#)[Business Studies](#)[Economics](#)[Management](#)[HR Management](#)[Finance](#)[Income Tax](#)

Preparation Corner

[Company-Wise Recruitment Process](#)[Resume Templates](#)

[Companies](#)[Colleges](#)

Competitive Exams

[JEE Advanced](#)[UGC NET](#)[UPSC](#)[SSC CGL](#)[SBI PO](#)[SBI Clerk](#)[IBPS PO](#)[IBPS Clerk](#)

Free Online Tools

[Typing Test](#)[Image Editor](#)[Code Formatters](#)[Code Converters](#)[Currency Converter](#)[Random Number Generator](#)[Random Password Generator](#)

More Tutorials

[Software Development](#)[Software Testing](#)[Product Management](#)[Project Management](#)[Linux](#)[Excel](#)[All Cheat Sheets](#)[Recent Articles](#)

Write & Earn

[Write an Article](#)[Improve an Article](#)[Pick Topics to Write](#)[Share your Experiences](#)[Internships](#)

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved